



3·4주차 네트워크 스터디 - Mini IDS 보고서

20262161 허정우

1. 과제 개요

Scapy로 PCAP을 분석하는 Mini IDS를 구현하였습니다. Port Scan, ARP Spoofing 의 심 징후, 특정 패킷(DNS·ICMP·HTTP·SSH)의 세 기능을 모두 다루었습니다. 테스트 패킷 은 네트워크로 전송하지 않고 합성 PCAP 파일로 만들어 분석하였습니다.

2. 탐지 기준과 핵심 구현

기능	사용 필드	탐지 기준	출력
Port Scan	IP 출발지·목적지, TCP 목적지 포트·SYN 플래그, 시 각	같은 출발지와 목적지에 대해 10초 안에 서로 다른 포트 20 개 이상에 SYN 전송	PORT_SCAN 경고
ARP Spoofing	ARP Reply의 Sender IP·Sender MAC	이미 저장한 동일 IP의 MAC 이 다른 값으로 관찰됨	ARP_SPOOFING 의심 경고
특정 패킷	DNS 질의, ICMP Type, TCP 목적지 포트·플래그, Raw 데이터	DNS Query, ICMP Echo Request, 평문 HTTP GET, TCP 22번 SYN 구분	각 패킷의 INFO 기록

`mini_ids.py` 의 `process_packet()` 이 각 패킷을 세 탐지 함수에 전달합니다. Port Scan 탐지 기는 최근 10초의 목적지 포트 기록을 유지하고, ARP 탐지기는 이전 IP-MAC 대응 관계를 저장하여 새 값과 비교합니다. 특정 패킷 탐지기는 DNS·ICMP·HTTP·SSH 조건을 식별하여 **INFO** 로 기록합니다. 해당 프로토콜이 관찰되었다는 사실만으로 공격이라고 판단하지 않았습니다.

2.1 핵심 판단 코드

다음은 탐지 여부를 결정하는 조건을 간추려 표현한 코드입니다. 실제 함수의 전체 구현은 `mini_ids.py` 에 있습니다.

Port Scan: TCP 플래그에서 SYN이 켜지고 ACK가 꺼진 패킷만 기록합니다. 최근 10초 동안 같은 출발지·목적지 쌍에서 관찰된 서로 다른 목적지 포트의 개수를 계산합니다.

```

is_syn = bool(flags & 0x02)
is_ack = bool(flags & 0x10)
unique_ports = {port for _, port in recent}
scan_detected = (
    is_syn and not is_ack
    and len(unique_ports) >= PORT_SCAN_THRESHOLD
)

```

ARP Spoofing 의심: ARP Reply의 Sender IP를 기준으로 저장된 MAC과 새 Sender MAC을 비교합니다.

```

previous_mac = self.arp_table.get(sender_ip)
mac_changed = previous_mac is not None and previous_mac != sender_mac

```

특정 패킷: DNS Query와 ICMP Echo Request는 계층과 필드로 구분합니다. HTTP는 TCP 80번 포트의 평문 GET, SSH 연결 시도는 TCP 22번 포트의 SYN으로 구분합니다.

```

dns_query = DNS in packet and DNSQR in packet and int(packet[DNS].qr) == 0
icmp_echo = ICMP in packet and int(packet[ICMP].type) == 8
http_get = destination_port == 80 and payload.startswith(b"GET ")
ssh_syn = destination_port == 22 and bool(flags & 0x02) and not bool(flags & 0x10)

```

위 코드는 판단 원리를 설명하기 위한 요약입니다. 실제 함수에서는 각 계층의 존재 여부를 확인한 뒤 필드에 접근하고, 시간 창의 기록을 갱신하며, 중복 경고를 제어합니다.

3. 재현 방법

```

python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python3 generate_test_pcap.py
python3 mini_ids.py --pcap pcaps/mini_ids_test.pcap

```

`generate_test_pcap.py` 는 Ethernet 계층을 포함한 합성 패킷 26개를 `pcaps/mini_ids_test.pcap` 에 저장합니다. 이 중 서로 다른 목적지 포트로 향하는 TCP SYN 20개와 동일 IP에 서로 다른 MAC이 담긴 ARP Reply 2개가 경고 검증에 사용됩니다.

4. 실행 결과 및 이미지

4.1 Mini IDS 실행 결과

아래 실행 화면에서 26개 패킷의 분석 결과와 `logs/alerts.log` 기록 경로를 확인할 수 있습니다. 첫 번째 화면에는 네 종류의 정보성 이벤트와 ARP·Port Scan 경고가 함께 표시됩니다. 두 번째 화면은 경고 부분을 확대한 화면입니다.

```
분석 파일: pcaps/mini_ids_test.pcap
패킷 수: 26
[INFO][DNS_QUERY] 192.0.2.10 → 192.0.2.53, 도메인=example.test
[INFO][ICMP_ECHO_REQUEST] 192.0.2.10 → 192.0.2.20, id=2026, seq=1
[INFO][HTTP_REQUEST] 192.0.2.10 → 192.0.2.80:80, GET /index.html HTTP/1.1
[INFO][SSH_CONNECTION_ATTEMPT] 192.0.2.10 → 192.0.2.22:22
[ALERT][ARP_SPOOFING] IP=192.0.2.1, 기존 MAC=aa:aa:aa:aa:aa:aa, 새 MAC=cc:cc:cc:cc:cc:cc
[INFO][SSH_CONNECTION_ATTEMPT] 192.0.2.50 → 192.0.2.100:22
[ALERT][PORT_SCAN] 출발지=192.0.2.50, 목적지=192.0.2.100, 10초 내 서로 다른 포트=20개
분석 완료. 결과 로그: logs/alerts.log
```

```
[INFO][DNS_QUERY] 192.0.2.10 → 192.0.2.53, 도메인=example.test
[INFO][ICMP_ECHO_REQUEST] 192.0.2.10 → 192.0.2.20, id=2026, seq=1
[INFO][HTTP_REQUEST] 192.0.2.10 → 192.0.2.80:80, GET /index.html HTTP/1.1
[INFO][SSH_CONNECTION_ATTEMPT] 192.0.2.10 → 192.0.2.22:22
[ALERT][ARP_SPOOFING] IP=192.0.2.1, 기존 MAC=aa:aa:aa:aa:aa:aa, 새 MAC=cc:cc:cc:cc:cc:cc
[INFO][SSH_CONNECTION_ATTEMPT] 192.0.2.50 → 192.0.2.100:22
[ALERT][PORT_SCAN] 출발지=192.0.2.50, 목적지=192.0.2.100, 10초 내 서로 다른 포트=20개
```

4.2 Port Scan 탐지 근거

첫 번째 Wireshark 화면은 합성 PCAP의 전체 26개 패킷을 보여 줍니다. 두 번째 화면에서는 `ip.src == 192.0.2.50 && ip.dst == 192.0.2.100 && tcp.flags.syn == 1 && tcp.flags.ack == 0` 필터를 적용하여 20개의 SYN 패킷을 확인하였습니다. 목적지 포트가 21, 22, 23부터 8443까지 서로 다르고, 패킷 시각은 2.0초부터 5.8초까지 분포합니다. 따라서 10초 이내에 서로 다른 포트 20개에 접근했다는 탐지 기준을 충족합니다.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	aa:aa:aa:aa:aa:aa	Broadcast	ARP	42	192.0.2.1 is at aa:aa:aa:aa:aa:aa
2	0.200000	192.0.2.10	192.0.2.53	DNS	72	Standard query 0x0000 A example.test
3	0.400000	192.0.2.10	192.0.2.20	ICMP	50	Echo (ping) request id=0x07ea, seq=1/256, ttl=64 (no response found!)
4	0.600000	192.0.2.10	192.0.2.80	HTTP	102	GET /index.html HTTP/1.1
5	0.800000	192.0.2.10	192.0.2.22	TCP	54	51001 → 22 [SYN] Seq=0 Win=8192 Len=0
6	1.000000	SiliconLabor_cc:cc:...	Broadcast	ARP	42	192.0.2.1 is at cc:cc:cc:cc:cc:cc (duplicate use of 192.0.2.1 detected!)
7	2.000000	192.0.2.50	192.0.2.100	TCP	54	40000 → 21 [SYN] Seq=0 Win=8192 Len=0
8	2.200000	192.0.2.50	192.0.2.100	TCP	54	40001 → 22 [SYN] Seq=0 Win=8192 Len=0
9	2.400000	192.0.2.50	192.0.2.100	TCP	54	40002 → 23 [SYN] Seq=0 Win=8192 Len=0
10	2.600000	192.0.2.50	192.0.2.100	TCP	54	40003 → 25 [SYN] Seq=0 Win=8192 Len=0
11	2.800000	192.0.2.50	192.0.2.100	TCP	54	40004 → 53 [SYN] Seq=0 Win=8192 Len=0
12	3.000000	192.0.2.50	192.0.2.100	TCP	54	40005 → 80 [SYN] Seq=0 Win=8192 Len=0
13	3.200000	192.0.2.50	192.0.2.100	TCP	54	40006 → 110 [SYN] Seq=0 Win=8192 Len=0
14	3.400000	192.0.2.50	192.0.2.100	TCP	54	40007 → 135 [SYN] Seq=0 Win=8192 Len=0
15	3.600000	192.0.2.50	192.0.2.100	TCP	54	40008 → 139 [SYN] Seq=0 Win=8192 Len=0
16	3.800000	192.0.2.50	192.0.2.100	TCP	54	40009 → 143 [SYN] Seq=0 Win=8192 Len=0
17	4.000000	192.0.2.50	192.0.2.100	TCP	54	40010 → 443 [SYN] Seq=0 Win=8192 Len=0
18	4.200000	192.0.2.50	192.0.2.100	TCP	54	40011 → 445 [SYN] Seq=0 Win=8192 Len=0
19	4.400000	192.0.2.50	192.0.2.100	TCP	54	40012 → 993 [SYN] Seq=0 Win=8192 Len=0
20	4.600000	192.0.2.50	192.0.2.100	TCP	54	40013 → 995 [SYN] Seq=0 Win=8192 Len=0
21	4.800000	192.0.2.50	192.0.2.100	TCP	54	40014 → 1433 [SYN] Seq=0 Win=8192 Len=0
22	5.000000	192.0.2.50	192.0.2.100	TCP	54	40015 → 3306 [SYN] Seq=0 Win=8192 Len=0
23	5.200000	192.0.2.50	192.0.2.100	TCP	54	40016 → 3389 [SYN] Seq=0 Win=8192 Len=0
24	5.400000	192.0.2.50	192.0.2.100	TCP	54	40017 → 5432 [SYN] Seq=0 Win=8192 Len=0
25	5.600000	192.0.2.50	192.0.2.100	TCP	54	40018 → 8080 [SYN] Seq=0 Win=8192 Len=0
26	5.800000	192.0.2.50	192.0.2.100	TCP	54	40019 → 8443 [SYN] Seq=0 Win=8192 Len=0

ip.src == 192.0.2.50 && ip.dst == 192.0.2.100 && tcp.flags.syn == 1 && tcp.flags.ack == 0						
No.	Time	Source	Destination	Protocol	Length	Info
7	2.000000	192.0.2.50	192.0.2.100	TCP	54	40000 → 21 [SYN] Seq=0 Win=8192 Len=0
8	2.200000	192.0.2.50	192.0.2.100	TCP	54	40001 → 22 [SYN] Seq=0 Win=8192 Len=0
9	2.400000	192.0.2.50	192.0.2.100	TCP	54	40002 → 23 [SYN] Seq=0 Win=8192 Len=0
10	2.600000	192.0.2.50	192.0.2.100	TCP	54	40003 → 25 [SYN] Seq=0 Win=8192 Len=0
11	2.800000	192.0.2.50	192.0.2.100	TCP	54	40004 → 53 [SYN] Seq=0 Win=8192 Len=0
12	3.000000	192.0.2.50	192.0.2.100	TCP	54	40005 → 80 [SYN] Seq=0 Win=8192 Len=0
13	3.200000	192.0.2.50	192.0.2.100	TCP	54	40006 → 110 [SYN] Seq=0 Win=8192 Len=0
14	3.400000	192.0.2.50	192.0.2.100	TCP	54	40007 → 135 [SYN] Seq=0 Win=8192 Len=0
15	3.600000	192.0.2.50	192.0.2.100	TCP	54	40008 → 139 [SYN] Seq=0 Win=8192 Len=0
16	3.800000	192.0.2.50	192.0.2.100	TCP	54	40009 → 143 [SYN] Seq=0 Win=8192 Len=0
17	4.000000	192.0.2.50	192.0.2.100	TCP	54	40010 → 443 [SYN] Seq=0 Win=8192 Len=0
18	4.200000	192.0.2.50	192.0.2.100	TCP	54	40011 → 445 [SYN] Seq=0 Win=8192 Len=0
19	4.400000	192.0.2.50	192.0.2.100	TCP	54	40012 → 993 [SYN] Seq=0 Win=8192 Len=0
20	4.600000	192.0.2.50	192.0.2.100	TCP	54	40013 → 995 [SYN] Seq=0 Win=8192 Len=0
21	4.800000	192.0.2.50	192.0.2.100	TCP	54	40014 → 1433 [SYN] Seq=0 Win=8192 Len=0
22	5.000000	192.0.2.50	192.0.2.100	TCP	54	40015 → 3306 [SYN] Seq=0 Win=8192 Len=0
23	5.200000	192.0.2.50	192.0.2.100	TCP	54	40016 → 3389 [SYN] Seq=0 Win=8192 Len=0
24	5.400000	192.0.2.50	192.0.2.100	TCP	54	40017 → 5432 [SYN] Seq=0 Win=8192 Len=0
25	5.600000	192.0.2.50	192.0.2.100	TCP	54	40018 → 8080 [SYN] Seq=0 Win=8192 Len=0
26	5.800000	192.0.2.50	192.0.2.100	TCP	54	40019 → 8443 [SYN] Seq=0 Win=8192 Len=0

4.3 ARP 변경 탐지 근거

arp.opcode == 2 && arp.src.proto_ipv4 == 192.0.2.1 필터로 같은 Sender IP를 가진 ARP Reply 두 개를 확인하였습니다. 첫 번째 패킷은 192.0.2.1 → aa:aa:aa:aa:aa:aa, 두 번째 패킷은 192.0.2.1 → cc:cc:cc:cc:cc:cc 으로 표시됩니다. 기존 IP-MAC 대응과 다른 값이 관찰되어 Mini IDS가 ARP Spoofing 의심 경고를 출력하였습니다. 이 경고만으로 실제 공격 여부가 확정되지는 않습니다.

arp.opcode == 2 && arp.src.proto_ipv4 == 192.0.2.1						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	aa:aa:aa:aa:aa:aa	Broadcast	ARP	42	192.0.2.1 is at aa:aa:aa:aa:aa:aa
6	1.000000	SiliconLabor_cc:cc:...	Broadcast	ARP	42	192.0.2.1 is at cc:cc:cc:cc:cc:cc (duplicate use of 192.0.2.1 detected!)

Info
192.0.2.1 is at aa:aa:aa:aa:aa:aa
192.0.2.1 is at cc:cc:cc:cc:cc:cc (duplicate use of 192.0.2.1 detected!)

4.4 특정 패킷 탐지 결과

4.1의 터미널 화면에서 `example.test` DNS 질의, ICMP Echo Request(`id=2026`, `seq=1`), `GET /index.html HTTP/1.1` 요청, TCP 22번 포트 연결 시도를 확인하였습니다. Wireshark의 전체 패킷 화면에서도 DNS·ICMP·HTTP 및 22번 포트 SYN 패킷이 나타납니다. 이 항목들은 공격 경고가 아닌 프로토콜 식별 결과입니다.

5. 오탐·미탐과 개선점

실제 환경에서 오탐, 미탐을 실험하진 않았지만, 다음과 같은 내용을 정리 할 수 있었습니다.

규칙	오탐 가능성(False Positive)	미탐 가능성(False Negative)	개선 방향
Port Scan	관리자의 정상적인 다중 포트 점검	10초보다 느리거나 20개 미만 포트를 검사하는 스캔	허용 IP와 장·단기 시간 창을 함께 사용
ARP 변경	공유기 교체·이중화 전환에 따른 정상 MAC 변경	첫 관찰값부터 잘못된 MAC인 경우	신뢰한 게이트웨이 MAC을 사전 등록하고 변경 이력 확인
특정 패킷	프로토콜 발견을 공격으로 오해할 수 있음	비표준 포트 사용 또는 HTTPS 암호화로 평문 내용 미확인	INFO와 ALERT를 분리하고 서비스별 규칙 보강

6. 결론

PCAP 26개를 분석하여 세 가지 탐지 기능을 검증하였습니다. 동일한 출발지에서 짧은 시간에 서로 다른 포트 20개로 향하는 SYN 패킷을 Port Scan으로 탐지하였고, 동일한 IP의 ARP MAC 변경을 의심 징후로 기록하였습니다. DNS·ICMP·HTTP·SSH 패킷은 정보성 이벤트로 구분하였습니다. 실제 환경에 적용하려면 정상 장비 변경과 관리 작업으로 인한 오탐, 느린 스캔과 초기 기준값 오염으로 인한 미탐을 줄이기 위한 추가 검증이 필요합니다.